

7/24/23, 3:07 PM

Codio - Third-Party Packages

CodeProjectFileEditFindViewToolsEducationHelpRunProject index (stand)Debug

Filetree x

SANDIPAN-DEY

Third-Party Packages

Third-Party Packages (master)

example1

settings

Terminal x

codio@thena:light-nevada-lopez:~/workspace\$

Guide x

Collapse

Uninstalling Packages

Uninstalling Packages

If `python3 -m pip install` is used to install a package, it stands to reason that `python3 -m pip uninstall` is how you remove it. We don't need Beautiful Soup right now, so let's uninstall it. Make sure that you are in the `example1` directory and you have activated the virtual environment.

```
cd example1
source example-venv/bin/activate
```

Now you are ready to uninstall the package.

```
python3 -m pip uninstall beautifulsoup4
```

Python is going to ask if you are really sure. Enter `y` in the terminal and press `ENTER`. Look at all of the installed packages in your virtual environment to verify the package is no longer there.

```
python3 -m pip list
```

Python asks that you verify your intention to remove a package. That is a handy way of making sure you don't accidentally remove an essential package. If you are sure you don't need the package, that extra check becomes a minor annoyance. You can uninstall packages while suppressing the verification.

```
python3 -m pip uninstall scipy -y
```

Try this variation:

- In the file tree, open `example1`. Then open `example-venv`. Under `lib` open `python3.6` and `site-packages`. You should see that Beautiful Soup and Scipy are no longer there.

Removing Packages and their Dependencies

When installing a package, `pip` will automatically install the dependencies. However, `pip` will not remove dependencies when uninstalling. For example, the `scipy` package has `numpy` as a dependency. So does `pandas`. If `pip` removed `numpy` when we uninstalled `scipy`, then `pandas` would no longer work as expected. To see the dependencies for a package use the `show` command.

```
python3 -m pip show pandas
```

Look under the `Requires:` line and you will see all of the dependencies for `pandas`. If you want to truly remove `pandas`, you need to remove it and all of its dependencies.

```
Name: pandas
Version: 1.1.5
Summary: Powerful data structures for data analysis, time series, and statistics
Home-page: https://pandas.pydata.org
Author: None
Author-email: None
License: BSD
Location: /home/codio/workspace/example1/example-venv/lib/python3.6/site-packages
Requires: pytz, python-dateutil, numpy
```

Let's remove `pandas` and all of its dependencies by listing them out in the same `uninstall` command. We will definitely want to use the `-y` flag so we don't have to keep approving each package removal.

```
python3 -m pip uninstall pandas pytz python-dateutil numpy -y
```

It is important to make sure that none of the other packages require the same dependencies as `pandas`. We could break our environment if we are not careful. One of the nice things about virtual environments is that they are fungible. Let's say we don't want `pandas` anymore, but we don't want to take the time to check the dependencies of all of the other packages. Create a second virtual environment. Use the `requirements.txt` file (minus `pandas`, of course) from the first virtual environment to install what you need. Then delete the first virtual environment. This is much easier and faster.

Try this variation:

- In the file tree, open `example1`. Then open `example-venv`. Under `lib` open `python3.6` and `site-packages`. You should see that `pandas` and all of its dependencies are no longer there.

Removing Packages with the Requirements File

Just as you can install a set of packages with the requirements file, you can uninstall all of those same packages with `requirements.txt`. Let's first deactivate the virtual environment for `example1` and then activate `example2`.

```
deactivate
cd ../example2
source venv/bin/activate
```

A requirements file does not exist for `example2`, so let's make one real quick.

```
python3 -m pip freeze > requirements.txt
```

Use the `uninstall` command along with `-r requirements.txt` to indicate the name of the requirements file. Again, using the `-y` flag will keep from having to confirm each package removal.

```
python3 -m pip uninstall -r requirements.txt -y
```

Finally, let's list out the packages in the virtual environment. You should not see `beautifulsoup4`, `scipy`, `pandas`, `numpy`, etc.

```
python3 -m pip list
```

Try this variation:

- In the file tree, open `example2`. Then open `venv`. Under `lib`, open `python3.6` and `site-packages`. You should see only the packages used on this system.

Reading Question

What is the **most important** difference between installing and uninstalling packages with `pip`?

☐ You can use `requirements.txt` to install packages but not remove them.

☐ Installing packages does not include their dependencies. Removing packages also uninstalls their dependencies.

☐ You have to confirm your choice when installing packages.

☒ Installing packages also includes their dependencies. Removing packages does not uninstall dependencies.

Installing and removing packages are processes that tend to mirror each other. However, when you remove a package, you do not remove any of its dependencies. Installing a package will also include the dependencies.

Check Me

Next